

DashyDashboard

Troubleshooting Guide

ASP.NET 7 (net7.0) | IIS | Windows Authentication | SQL Server

Target SQL Server: clipvwbpod02 | Database: ClientsAppAttestation

June 2026

How to use this guide. Find the symptom that matches what you see. Each entry describes exactly what error or behaviour you observe, why it happens, and the exact steps to fix it. If the fix involves editing a file, the exact path and the exact text to type are given.

Contents

1	App Refuses to Start — “Connection string not configured”	2
2	App Returns 500 on Every Page — No Useful Message	2
3	Browser Shows “HTTP Error 502.5 — ANCM Out-Of-Process Startup Failure”	3
4	App Starts but Returns 401 Unauthorized on Every Request	3
5	Users Can Log In but See an Empty Dashboard (No Data)	4
6	Users Can Log In but See “Access Denied” or Wrong Role	4
7	SQL Server Connection Fails — “Login failed” or “Cannot open database”	5
8	SSL / TLS Error Connecting to SQL Server	5
9	The Publish Command Fails with an npm Error	6
10	The publish Command Fails with a dotnet SDK Version Error	7
11	dotnet ef Command Not Found	7
12	Demo Users Appeared in the Production Database	8
13	IIS Shows the Site but the React App Does Not Load	8
14	A Specific Page Returns 404 After Reload	9
15	Rate Limit Error — HTTP 429	9
16	Changes to appsettings.Production.json Are Not Being Picked Up	9
17	Quick Diagnostic Checklist	10

App Refuses to Start — “Connection string not configured”

Symptom. The site returns a 500 error immediately. The IIS logs or stdout log show:

```
InvalidOperationException: Connection string 'Default' is not configured.  
Set it in appsettings.Production.json or via environment variables.
```

Why this happens. The app looks for the key `ConnectionStrings:Default` and throws on startup if it is missing or blank. This key is only in `appsettings.Production.json`, which is only loaded when `ASPNETCORE_ENVIRONMENT=Production`.

Fix — check these in order:

1. Open `C:\inetpub\DashyDashboard\appsettings.Production.json` in Notepad. Confirm the file contains a `ConnectionStrings` block. If the file is empty, missing, or the block is absent, paste in the full content from the Production Deployment Guide, Section 4.
2. Confirm `ASPNETCORE_ENVIRONMENT` is set to `Production`. Open `C:\inetpub\DashyDashboard\web.config` in Notepad. Look for:

```
<environmentVariable name="ASPNETCORE_ENVIRONMENT"  
                    value="Production" />
```

If it is absent, add it inside the `<environmentVariables>` block (see the Production Deployment Guide, Section 3 for the exact structure).

3. If you set `ASPNETCORE_ENVIRONMENT` in IIS Manager (not in `web.config`), confirm it is spelled exactly `ASPNETCORE_ENVIRONMENT` (all caps, underscore between words). A single character difference means the app loads the wrong settings file.
4. After any change, recycle the application pool. Open IIS Manager, click **Application Pools**, right-click **DashyDashboard**, click **Recycle**. Then browse to the site again.

App Returns 500 on Every Page — No Useful Message

Symptom. The browser shows a generic error page. The response body (if any) contains only a status code and no stack trace.

Why this happens (intentional behaviour). The app never leaks stack traces in production. The global exception handler in `Program.cs` returns a JSON problem response with no internal details. This is by design. You must read the actual error from the server-side logs.

How to read the logs:

1. Open IIS Manager.
2. In the left panel, click on the **DashyDashboard** site.
3. In the centre panel, double-click **Logging**. Note the **Directory** field value (e.g. `C:\inetpub\logs\LogFiles`).
4. Open File Explorer and navigate to that directory. Open the most recently modified `.log` file in Notepad. The last few lines will show the HTTP status codes and error entries.

To enable stdout logging for deeper errors:

1. Open `C:\inetpub\DashyDashboard\web.config` in Notepad.
2. Change `stdoutLogEnabled="false"` to `stdoutLogEnabled="true"`.
3. Create the logs folder if it does not exist: open a Command Prompt as Administrator and run:

```
mkdir C:\inetpub\DashyDashboard\logs
```

4. Press **Ctrl + S** to save `web.config`. Browse to the site to reproduce the error.
5. Open `C:\inetpub\DashyDashboard\logs` and read the `stdout_*.log` file.
6. **Turn stdout logging off again after you are done** (`stdoutLogEnabled="false"`). Leav-

ing it on will grow the log files indefinitely.

Browser Shows “HTTP Error 502.5 — ANCM Out-Of-Process Startup Failure”

Symptom. The IIS default error page shows: *HTTP Error 502.5 - ANCM Out-Of-Process Startup Failure* or *HTTP Error 500.30 - ASP.NET Core app failed to start*.

Why this happens. The ASP.NET Core module (ANCM) started the `DashyDashboard.Api.exe` process but the process crashed or failed to bind a port before IIS timed out. Common causes: the .NET 7 runtime is not installed, the connection string is missing, or the binary is for the wrong architecture.

Fix — work through these in order:

1. **Confirm the .NET 7 runtime is installed.** Open a Command Prompt on the IIS server and run:

```
dotnet --list-runtimes
```

You must see a line containing `Microsoft.AspNetCore.App 7.0`. If you do not, install the .NET 7 Hosting Bundle (Production Deployment Guide, Section 2) and then run `iisreset`.

2. **Check the connection string.** A missing connection string causes an immediate crash. See Section 1.
3. **Enable stdout logging and read the actual error message.** See Section 2 for how to enable it. The stdout log will show the exact exception that killed the process.
4. **Check the Windows Event Log.** Press **Win + R**, type `eventvwr`, press **Enter**. Expand **Windows Logs → Application**. Look for Error entries with source **IIS AspNetCore Module** or **DashyDashboard**. The event message usually contains the exact startup error.

App Starts but Returns 401 Unauthorized on Every Request

Symptom. Every page returns a **401 Unauthorized** response. The browser may show a login popup that keeps rejecting credentials, or the app immediately shows an error page.

Why this happens. Windows Authentication is not correctly configured on the IIS site. The most common causes:

- Anonymous Authentication is still enabled (it should be disabled).
- Windows Authentication is disabled (it should be enabled).
- The Negotiate provider is missing from the Windows Authentication providers list.
- The Windows Authentication feature was never installed on Windows Server.

Fix:

1. Open IIS Manager. In the left panel, click on the **DashyDashboard** site. In the centre panel, double-click **Authentication**.
2. Confirm the state of each entry:

Authentication type	Required state
Anonymous Authentication	Disabled
Windows Authentication	Enabled

3. If Anonymous is enabled, click it and click **Disable** in the Actions panel. If Windows Authentication is disabled, click it and click **Enable**.
4. With Windows Authentication selected, click **Providers...** Confirm **Negotiate** is in the list and is above **NTLM**. If Negotiate is missing, click **Add**, select **Negotiate**, click **OK**, then click **Move Up** to put it above **NTLM**. Click **OK**.
5. If Windows Authentication is greyed out and cannot be enabled, the Windows feature is

not installed. Go to Server Manager → Add Roles and Features → Web Server (IIS) → Web Server → Security and tick **Windows Authentication**. Install it. Then return to IIS Manager and enable it.

Users Can Log In but See an Empty Dashboard (No Data)

Symptom. Windows Authentication works and users reach the app. But the dashboard shows no cycles, no tools, or empty lists. There are no JavaScript errors in the browser console.

Why this happens. The database tables exist but are empty (no data was seeded), or the migrations have not been run so the tables do not exist yet.

Fix:

1. **Confirm the database migrations have been applied.** From a machine with the .NET 8 SDK that can reach clipvwbpod02, run:

```
cd DashyDashboard/DashyDashboard.Api
dotnet ef migrations list --connection "Server=clipvwbpod02;Database=
↳ ClientsAppAttestation;Integrated Security=True;Encrypt=True;
↳ TrustServerCertificate=False;"
```

Each migration in the list should show **(Applied)**. If any show **(Pending)**, run:

```
dotnet ef database update --connection "Server=clipvwbpod02;Database=
↳ ClientsAppAttestation;Integrated Security=True;Encrypt=True;
↳ TrustServerCertificate=False;"
```

2. **Confirm the app pool identity can read from the database.** Open SQL Server Management Studio on a machine that can reach clipvwbpod02. Run:

```
USE ClientsAppAttestation;
SELECT name FROM sys.database_principals
WHERE name = 'IIS APPPOOL\DashyDashboard';
```

If no row is returned, the login was never created. Follow Step 6 in the Windows Authentication Setup section of the Production Deployment Guide.

3. **Check that production data exists.** The app ships with no built-in production data. An administrator must create at least one Cycle via the admin interface before any user will see work to do. Demo data (Bruce Banner etc.) must not be used in production — see Section 12 of this guide.

Users Can Log In but See “Access Denied” or Wrong Role

Symptom. A user who should be a Manager sees the Agent view. Or a user who should be an Admin has no admin menu. Or a user who should be a GFH/IFH sees nothing at all.

Why this happens. Manager status is derived at runtime from whether any other user has that person’s AssociateId as their ManagerId value. GFH/IFH/Admin roles require a row in the SuperUsers table.

Known blocker: the SuperUsers table was added in a later migration. If the database was created from an older migration, the table may not exist. Also, the current code stores AssociateId as int, which means any user with a non-numeric ID is silently unresolvable regardless of role. These are tracked blockers in the codebase.

Fix:

1. **Check that all migrations are applied.** Run the dotnet ef migrations list command from Section 5. The migration AddSuperUsersAndLogs must show **(Applied)**. If it shows **(Pending)**, run dotnet ef database update.

2. **Check the SuperUsers table for the affected user.** In SQL Server Management Studio, run:

```
SELECT * FROM dbo.SuperUsers
WHERE AssociateID = 'their-associate-id';
```

If no row exists, the user has not been granted a SuperUser role. Ask the administrator to insert a row.

3. **Check Manager status.** To diagnose why a user is or is not seen as a manager, run:

```
SELECT AssociateID, FirstName, LastName, ManagerId
FROM dbo.Users
WHERE ManagerId = 'the-suspected-manager-associate-id';
```

If no rows are returned, that person has no direct reports in the database and will not see the Manager view. Ensure the direct reports' ManagerId column is correctly populated.

SQL Server Connection Fails — “Login failed” or “Cannot open database”

Symptom. The app starts but every API call returns 500. The stdout log or Windows Event Log shows an error such as:

```
Microsoft.Data.SqlClient.SqlException: Login failed for user
'IIS APPPOOL\DashyDashboard'.
```

or:

```
Cannot open database "ClientsAppAttestation" requested
by the login.
```

Why this happens. The IIS application pool identity has not been granted access to the ClientsAppAttestation database on clipvwbpod02.

Fix:

1. Ask the DBA to run this on clipvwbpod02:

```
USE ClientsAppAttestation;

CREATE LOGIN [IIS APPPOOL\DashyDashboard]
FROM WINDOWS
WITH DEFAULT_DATABASE = ClientsAppAttestation;

CREATE USER [IIS APPPOOL\DashyDashboard]
FOR LOGIN [IIS APPPOOL\DashyDashboard];

ALTER ROLE db_datareader
ADD MEMBER [IIS APPPOOL\DashyDashboard];

ALTER ROLE db_datawriter
ADD MEMBER [IIS APPPOOL\DashyDashboard];
```

2. If the SQL runs successfully but login still fails, confirm the IIS application pool identity is the default (not a custom service account). Open IIS Manager → Application Pools → right-click **DashyDashboard** → **Advanced Settings...** Look at the **Identity** row. If it is not **ApplicationPoolIdentity**, the account name in the SQL above is wrong. The account to grant access to is whatever account name appears in the **Identity** row.

SSL / TLS Error Connecting to SQL Server

Symptom. The stdout log or EF migrations output shows:

```
A connection was successfully established with the server, but then an error occurred
```

```
during the login process. (provider: SSL Provider, error: 0 - The certificate chain
was issued by an authority that is not trusted.)
```

Why this happens. The connection string contains `Encrypt=True;TrustServerCertificate=False;`. The SQL Server on `clipvwbpod02` has a certificate that is not trusted by the machine running the app.

Fix options (choose one):

1. **(Preferred) Ask the DBA to install a certificate on clipvwbpod02** that is signed by an internal CA trusted by all servers on the network. This is the correct fix for a production environment.
2. **(Temporary, for migration only) Change the migration command** to trust the certificate just for that one command: Replace `TrustServerCertificate=False` with `TrustServerCertificate=True` in the `-connection` argument. Do **not** make this change in `appsettings.Production.json`.
3. **(Not recommended) Change the app's connection string.** Changing `TrustServerCertificate` to `True` in `appsettings.Production.json` disables certificate validation for every database connection the app makes. Do this only if the DBA confirms it is acceptable for the network environment.

The Publish Command Fails with an npm Error

Symptom. Running `dotnet publish ... -c Release` fails with an error referencing `npm` or `node`:

```
error MSB3073: The command "npm ci --no-audit --no-fund" exited with code 1.
```

or:

```
'npm' is not recognized as an internal or external command
```

Why this happens. The publish step automatically runs `npm ci` and `npm run build` to compile the React frontend. Node.js and npm must be installed and on the system `PATH` on the build machine.

Fix:

1. Open a new Command Prompt and run:

```
node --version
npm --version
```

If either fails, Node.js is not installed or not on the `PATH`. Install Node.js 18 LTS (Production Deployment Guide, Section 2).

2. If Node is installed but the command still fails, the problem may be a missing `package-lock.json` file. Navigate to the frontend folder:

```
cd DashyDashboard/DashyDashboard.Frontend
```

Check whether `package-lock.json` exists:

```
dir package-lock.json
```

If it is missing, generate it:

```
npm install
```

Then go back to the repo root and run the publish command again.

3. If `npm ci` fails with a dependency error or audit error, try running it manually to see the full error:

```
cd DashyDashboard/DashyDashboard.Frontend
```

```
npm ci
```

Read the error output and resolve the specific dependency conflict.

The publish Command Fails with a dotnet SDK Version Error

Symptom. Running the publish command fails with an error such as:

```
MSB1009: Project file does not exist.
```

or:

```
error NU1105: Unable to find project information for ...
```

or the build exits with errors about missing targets.

Why this happens. The app targets `net7.0`. The build machine needs a .NET SDK that can compile `net7.0` targets (SDK 7.0.x minimum; SDK 8.0.x also works).

Fix:

1. Run this to see what SDK versions are installed:

```
dotnet --list-sdks
```

2. If the output is empty or shows only SDK 6.x or earlier, install the .NET 7 SDK. See the Production Deployment Guide, Section 2.
3. If you see SDK 7.x or 8.x listed, the SDK is fine. The error is probably something else — enable verbose output:

```
dotnet build -v detailed 2>&1 | findstr /i "error"
```

Read the first error line and resolve it specifically.

dotnet ef Command Not Found

Symptom. Running `dotnet ef database update` prints:

```
Could not execute because the specified command or file was not found.
Possible reasons for this include:
  * You intended to execute a .NET program
```

Why this happens. The EF Core CLI (`dotnet-ef`) is a global tool that must be installed separately. It is not bundled with the .NET SDK.

Fix:

1. Install the EF Core CLI tool:

```
dotnet tool install --global dotnet-ef
```

2. After installation, open a **new** Command Prompt window (the `PATH` is updated for new windows only).
3. Run `dotnet ef -version` to confirm it is installed.
4. Run the migration command again.

If you see the error again even after installing, your `PATH` does not include the dotnet tools directory. Open a Command Prompt and run:

```
set PATH=%PATH%;%USERPROFILE%\dotnet\tools
```

Then try again in the same window.

Demo Users Appeared in the Production Database

Symptom. Users with names such as **Bruce Banner**, **Diana Prince**, or **Selina Kyle** appear in the live database.

Why this happened. The seed code ran in production. This happens only when **both** of these are true at the same time: `ASPNETCORE_ENVIRONMENT` equals `Development`, and `DeveloperMode:EnableDemoSeed` equals `true`. One or both of these were true when the app started.

Fix:

1. **Immediately** open `C:\inetpub\DashyDashboard\appsettings.Production.json` in Notepad. Confirm all three `DeveloperMode` values are `false`.
2. Confirm `ASPNETCORE_ENVIRONMENT` is `Production` in `web.config`, not `Development`.
3. Recycle the application pool.
4. Remove the demo rows from the database. Ask the DBA to run:

```
USE ClientsAppAttestation;

-- Remove all data created by the seed (check names/IDs carefully first)
DELETE FROM dbo.ToolCycleAttestation
WHERE AssociateID IN (
    SELECT AssociateID FROM dbo.Users
    WHERE FirstName IN ('Bruce', 'Diana', 'Selina', 'Clark', 'Tony',
                        'Natasha', 'Peter', 'Steve', 'Wanda',
                        'James', 'T', 'Challa', 'Carol')
);

DELETE FROM dbo.UserToolAccess
WHERE AssociateID IN (
    SELECT AssociateID FROM dbo.Users
    WHERE FirstName IN ('Bruce', 'Diana', 'Selina', 'Clark', 'Tony',
                        'Natasha', 'Peter', 'Steve', 'Wanda',
                        'James', 'T', 'Challa', 'Carol')
);

DELETE FROM dbo.Users
WHERE FirstName IN ('Bruce', 'Diana', 'Selina', 'Clark', 'Tony',
                    'Natasha', 'Peter', 'Steve', 'Wanda',
                    'James', 'T', 'Challa', 'Carol');
```

Review and confirm the rows before deleting. Run the `SELECT` part of each statement first to see exactly what will be deleted.

IIS Shows the Site but the React App Does Not Load

Symptom. Navigating to the site URL shows a blank white page, a directory listing, or the raw JSON from the API instead of the React app.

Why this happens. The React frontend was not compiled into the `wwwroot` folder, or the published output was copied incorrectly and `wwwroot/index.html` is missing.

Fix:

1. On the IIS server, open File Explorer and navigate to the site physical path (e.g. `C:\inetpub\DashyDashboard`).
2. Look for a folder named `wwwroot`. Inside it there must be an `index.html` file and an `assets` folder containing `.js` and `.css` files. If `wwwroot` is empty or missing, the React build output was not included.
3. Go back to the build machine and re-run the publish command (Production Deployment Guide, Section 7, Step 4). Do not skip or cancel the npm build step. Wait for the full Build

`succeeded.` message.

4. Copy the entire `publish\DashyDashboard.Api` folder to the server again, replacing all existing files.
5. Open a browser and navigate to the site. Press **Ctrl + Shift + I** to open developer tools. Click **Network** and press **F5** to reload. If you see `index.html` loading with a 200 status, the frontend is now being served correctly.

A Specific Page Returns 404 After Reload

Symptom. The app loads fine from the home page. But if a user navigates to a page (e.g. `/manager` or `/attestations`) and then presses **F5** (browser refresh), they get a 404 from IIS.

Why this happens. This is a single-page application (SPA). The routes like `/manager` exist only inside React. When the browser refreshes, it asks the server for `/manager` as if it were a real file. The server finds no file at that path and returns 404.

Fix: The app is configured to handle this automatically via:

```
app.MapFallbackToFile("index.html");
```

If you are still seeing 404 on refresh, the ANCM module is not routing requests to the app. Possible causes:

1. **The application pool is stopped.** Open IIS Manager → Application Pools. If **DashyDashboard** shows **Stopped**, right-click it and click **Start**.
2. **Static files are being served by IIS directly, bypassing the app.** This can happen if a `web.config` `<staticContent>` rule is intercepting the request before it reaches the .NET handler. Remove any custom `<staticContent>` or `<handlers>` rules from `web.config` that you may have added.
3. **Wrong hostingModel in web.config.** Confirm the `<aspNetCore>` element has `hostingModel="inprocess"` (not `outofprocess`).

Rate Limit Error — HTTP 429

Symptom. The app returns HTTP 429 Too Many Requests for a specific user or IP address.

Why this happens. The app enforces a fixed-window rate limit of 120 requests per minute, partitioned by Windows username. If a single user (or the same IP for unauthenticated requests) sends more than 120 requests within one 60-second window, further requests are rejected until the window resets.

What to check:

- This is expected behaviour, not a bug. A normal human user performing normal attestation work will not hit 120 requests per minute.
- If an automated process or integration is hitting the limit, it must be slowed down.
- If a legitimate user is hitting the limit because of a frontend bug (for example, an infinite retry loop), open browser developer tools, click **Network**, and look for the request that is being sent repeatedly. Find and fix the offending frontend code.

Changes to `appsettings.Production.json` Are Not Being Picked Up

Symptom. You edited `appsettings.Production.json` on the server but the app behaviour has not changed.

Why this happens. The app reads configuration files at startup. Changing the file after the app has started has no effect until the app restarts. Unlike `web.config` (which IIS monitors and reloads automatically), `appsettings.Production.json` is only read when the process starts.

Fix:

1. Confirm you saved the file. Open it in Notepad again to verify your changes are present.
2. Recycle the application pool. Open IIS Manager → Application Pools → right-click **DashyDashboard** → **Recycle**.
3. Browse to the site. Your changes should now be in effect.

Exception: If you set configuration values via IIS environment variables (in `web.config` or the Configuration Editor), those are *also* read at process start. Saving a change to `web.config` causes IIS to restart the worker process automatically, so environment variable changes in `web.config` do not need a manual recycle.

Quick Diagnostic Checklist

If you are unsure where a problem is, work through this checklist top to bottom. Each item is a yes/no question. The first “No” tells you where to look.

#	Check	If No, go to
1	Does <code>dotnet -list-runtimes</code> on the IIS server show <code>Microsoft.AspNetCore.App 7.0.x</code> ?	Section 3
2	Does <code>C:\inetpub\DashyDashboard\appsettings.Production.json</code> exist and contain <code>Server=clipvwbpod02</code> ?	Section 1
3	Is <code>ASPNETCORE_ENVIRONMENT</code> set to <code>Production</code> in <code>web.config</code> or IIS?	Section 1
4	Is Windows Authentication enabled and Anonymous Authentication disabled on the IIS site?	Section 4
5	Has IIS <code>APPPool\DashyDashboard</code> been granted <code>db_datareader</code> and <code>db_datawriter</code> on <code>ClientsAppAttestation</code> ?	Section 7
6	Do all migrations show (Applied) in <code>dotnet ef migrations list</code> ?	Section 5
7	Does <code>wwwroot/index.html</code> exist inside the site physical path?	Section 13
8	Are all three <code>DeveloperMode</code> flags false in <code>appsettings.Production.json</code> ?	Section 12